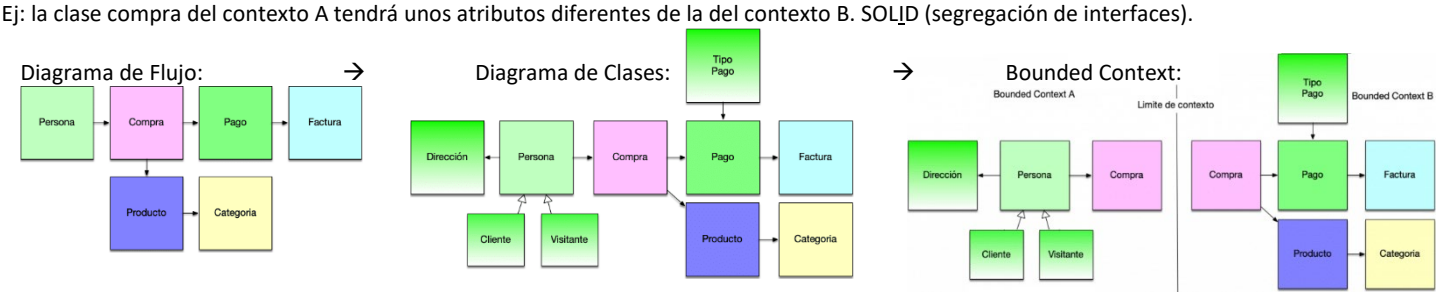
# 33 - BLOQUE 4 – TEMA 3 – MICROSERVICIOS

Los contenedores son ideales para las arquitecturas de µs. Son servicios que pueden ser desplegados de manera independiente y que se modelan alrededor de un dominio de negocio. µ no significa que sea muy pequeño, el tamaño lo dictará el servicio que preste. Son cajas negras que tienen una funcionalidad definida y producen un resultado. Son un tipo de SOA. No todo funciona mejor con µs. Muy débilmente acoplados. Granularidad muy fina (poca responsabilidad). Se comunican mediante protocolos ligeros (http). Equipos de desarrollo independientes. Podemos definirlos en diferentes lenguajes y con distintas BD. Cada µs puede tener su BD (100% independiente). Resiliencia, si falla un µs no se viene abajo toda la app.

**Nuevos retos:**
Independientemente desplegables. Distribución de responsabilidad. Independientemente escalables. Acoplamiento y Cohesión. Mezcla de tecnologías. Gobernanza: gestionar los µs es complejo, por eso hay orquestadores, patrones de diseño específicos, transaccionalidad, dependencias, etc. DDD (Domain Driven Design): intenta descubrir los conceptos del negocio desde varios puntos de vista, permitiéndonos definir servicios, entidades y repositorios.

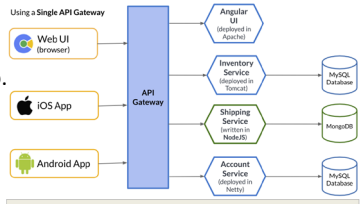
Bounded Context: patrón de diseño que divide el modelo en submodelos, especializados en áreas. Cada µs tendrá un Bounded Context asociado, se encargará de una funcionalidad del negocio, de un contexto, de un área de negocio. Es mejor segregar.
Ej: la clase compra del contexto A tendrá unos atributos diferentes de la del contexto B. SOLID (segregación de interfaces).



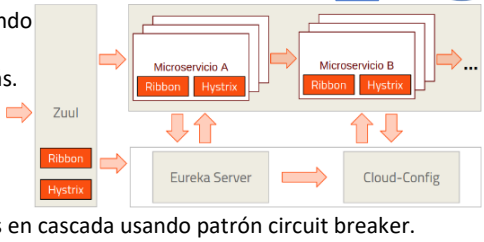

**Nuevos problemas:** latencia, fallos de red, consistencia de datos, sobrecarga de operaciones, resolución de problemas, muchas opciones a elegir...

**ARQUITECTURA**
**Comunicación entre µs:**

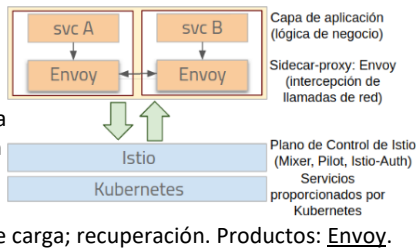
- Síncrona: mala, porque dependes del eslabón más débil.
- Asíncrona:
  - Petición-Respuesta: basado en colas. Es transaccional.
    Brokers de mensajes: intermediarios entre µs para una comunicación asíncrona (Colas de mensajes / Topics).
    Entrega garantizada. Confianza. Transaccionalidad.
    Productos: Apache Kafka, RabbitMQ, Apache ActiveMQ, Google Cloud Pub/Sub.
  - Dirigido por eventos: genero un evento con info y mis suscriptores (Topic) usan el evento y la info como inicio de su trabajo.
  - Datos compartidos: no es la más típica. Subes un fichero a la nube, notificas a los interesados y podrán consumirlo cuando lo necesiten.
    Pueden usarse volúmenes de un contenedor, NAS/S3 o BBDD en modelos con segregación de responsabilidad (CQRS).
- RPC/gRPC: a día de hoy de lo más usado. HTTP2. Protocol Buffer. Generación de código nativa.
- SOAP: muy usado en la AGE por el ENI.
- RMI: muy dependiente de la tecnología (Java), pero permite aprovechar el conocimiento previo del equipo.
- GraphQL: agregador de consultas con capacidades de filtrado. Resolver, Queries y Mutations.
- REST: a día de hoy de lo más usado. HTTP 1.1. XML+JSON. Generación de código con OpenAPI / Swagger.



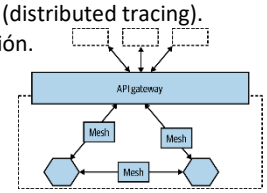

- **API Gateway**: v1.0. Puerta de entrada a los µs, aislándolos del exterior. Productos liberados por Netflix. Zuul es el primer producto que hace de proxy inverso, canalizando peticiones y ocultando la complejidad de los µs. Zuul se replica muchas veces para que no caiga todo en caso de fallo.
  API Gateway es un patrón de diseño que centraliza la lógica de acceso a los µs que están detrás.
  Esta versión tiene mucha complejidad para gestionar los µs.
  Patrón Circuit Breaker: Cuando un µs llama a otro y éste a otro, si alguno falla, hace que no se caiga todo. Permite definir política de reintentos y timeout. Corta el acceso al µs erróneo de cara a futuras llamadas por otros µs e implementa una política de restauración para ese µs.
  Zuul: API Gateway/Edge Service.
  Ribbon: librería para balancear peticiones de clientes.
  Hystrix: librería de errores en cascada usando patrón circuit breaker.
  Eureka: servidor para registrar µs y sus metadatos.



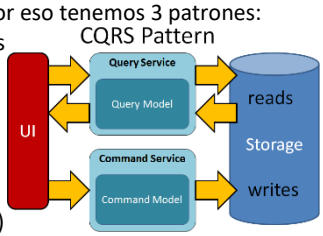

- **Service Mesh**: v2.0. Tecnología diseñada para abordar las complejidades de las arquitecturas de µs. Solo tienes que instalar el producto Istio (podría ir en un contenedor) y Envoy (que es el proxy, la implementación del patrón sidecar). Istio habla con los Envoy. Centraliza las peticiones de los proxys. Service Mesh = Istio + Envoy. Service Mesh es como un Framework, hace el trabajo sucio a los µs para solo preocuparte de la lógica de negocio, no de la arquitectura. Plano de control donde se solucionan problemas de chequeo de políticas de seguridad, métricas, etc. Los proxys se comunican entre ellos.
  Patrón sidecar: te ayuda, dialogando con otra parte de la arquitectura llamada Service Mesh.
  - Abstracción: abstrae a los desarrolladores de los problemas de arquitectura: discovery; balanceo de carga; recuperación. Productos: Envoy.
  - Centralización: se crea un punto centralizado para controlar las funciones de los µs en una aplicación. Productos: Istio, Linkerd.
  - Trazabilidad: centralizamos y mostramos trazas asociadas con una sola solicitud entre múltiples servicios: Distributed tracing; Monitorizar transacciones distribuidas; Análisis de Root Cause. Id único para cada traza. Productos: Jaeger, Spring Cloud Sleuth (distributed tracing).
  - Observabilidad: observaciones y telemetría sobre nuestros µs: Salud; Topología; Métricas; Dashboards; Configuración.
    Productos: elasticsearch (logs), Grafana (dashboard), kiali, Prometheus (métricas).
  - Seguridad: Cifrado; Autenticación; Autorizaciones. Productos: keycloak: gestión de identidades con SSO (Java).

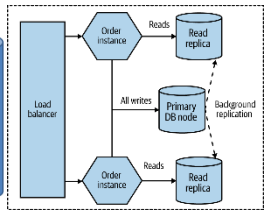



- Es bastante habitual combinar API Gateway y Service Mesh. →



**Bases de datos:** no podemos asegurar ACID por ser un sistema altamente distribuido. Por eso tenemos 3 patrones:




- **CQRS**: Command and Query Responsibility Segregation. CRUD: La mayor parte de las operaciones son de lectura (R). El resto son de creación (C), actualización (U), y borrado (D). Por eso dividimos el µs en 2: R y CUD.
- **2PC** (Two-Phase Commits) - Transacciones distribuidas: fase de Voting y fase de Commit. No suele usarse por ser muy costoso.
- **SAGA**: algoritmo que coordina múltiples cambios en estados, evitando bloquear recursos por largos periodos de tiempo. Separamos una LLT (Long Lived Transaction) en una secuencia de tx independientes. Si hay fallos podemos reintentar o hacer rollback. Se gobierna con Orquestador o Coreógrafo. Como es difícil hacer rollback, se guarda una tx compensatoria. En lugar de hacer rollback en todos los µs, mandamos a todos los µs la tx compensatoria.

**Frameworks**

● Java: Spring Cloud/Boot, Micronaut, Quarkus, Helidon, Microprofile (Jakarta). ● Python: Flask. ● JavaScript: NodeJS. ●.NET

**Orquestación**

Nodo: elemento del clúster que puede albergar pods en su interior.
Clúster: agrupación de nodos a orquestar. Al menos 1 master y 1 worker.
Pod: conjunto de uno o más contenedores.
Servicio: conjunto de pods a los que enviamos tráfico (ingress).
Volumen: directorio con datos accesibles a los contenedores del pod.
Namespace: clúster virtual o partición de nuestro clúster.
Productos: Docker Swarm y Kubernetes.

**Despliegue**

- Ejecución aislada: ejecuta los µs de forma aislada, con sus recursos computacionales, y que su ejecución no impacte a otros µs cercanos.
- Automatización: según crezca el número de µs, la automatización cobra mayor importancia (CI/CD, DevOps).
- IaC: representa la configuración de tu infraestructura para facilitar los despliegues. Se usa control de versiones.
- Despliegues Zero-Downtime: intenta desplegar µs sin tener downtime para los usuarios o para tu servicio.
- Estados deseados: declarativo. Usa una plataforma que mantenga los µs en un estado definido, lanzando nuevas instancias si fuese necesario.

**Modos de despliegue**

- Máquina física: la instancia del µs se ejecuta en una máquina física directamente, sin virtualización.
- Máquina virtual: se ejecuta en una máquina virtual.
- Contenedor: se ejecuta en un contenedor separado, en un host físico o virtual. Puede gestionarse por un orquestador como k8s (Kubernetes).
- Contenedor de Aplicaciones: el µs se ejecuta dentro de un contenedor de aplicaciones que administra otras aplicaciones. Tomcat , WebLogic.
- PaaS: instancias administradas, abstrayéndonos de la gestión inferior y que corren nuestros µs. Heroku, Google AppGengine, AWS Beanstalk.
- FaaS: la instancia del µs se despliega como una función (serverless) como AWS Lambda, Google Cloud Functions o Azure Functions.
- KaaS: servicio de orquestación de contenedores en la nube. Creas contenedores desde repositorios de imágenes y la infraestructura es administrada por el CSP (Proveedor de Servicios de Comunicaciones). Amazon EKS, Google GKE, Azure AKS.

**Tests**

- Unitarios (TDD): muy importantes en entornos CI/CD.
- Servicio: se prueba el µs como un todo. Stubs y Mocks.
- End-to-End: sistema completo.

**Observabilidad**

- Agregación de logs: cada µs tiene sus registros, pero conviene tenerlo todo recopilado.
- Agregación de métricas: se capturan constantes de los µs (cpu, ram, peticiones), pero también solicitudes, compras o indicadores del negocio.
- Trazas distribuidas: hay que saber qué ha ocurrido y no perder el rastro de una solicitud o una saga.
- Alertas: con esta información podemos lanzar alertas que incluso emitan eventos para modificar la infraestructura. Evitar la fatiga de alertas.
- Pruebas en producción: transacciones sintéticas, Canary, Tests A/B, Ejecuciones paralelas, Chaos Engineering.

**Seguridad**

Debe formar parte desde el inicio del desarrollo. Service Mesh ofrece muchas de estas funcionalidades pero hay que tenerlas siempre presentes.

- Mínimo privilegio posible y alcances limitados: esta cuenta de servicio solo puede acceder aquí. Este servicio se identifica con esta cuenta.
- Credenciales fuera del código, en el entorno o en Secrets Managers.
- Autenticaciones S2S, OAUTH2, TLS, JSON Web Token.

**Resiliencia**

**Timeouts**: tiempo permitido a una ejecución hasta que falle o tiempo que hay que esperar antes de decidir que ha ido mal.
**Reintentos**: si algo falla, no lo reintento inmediatamente. Aplico back-off exponencial y un número máximo de intentos.
**Circuit Breaker:** si hay un fallo con otro µs: se detecta, se dejan de enviar peticiones, se recupera y volvemos a enviarlo.
Estados básico: abierto y cerrado. Cerrado: cierra el acceso a un µs si después de aplicar la política de reintentos se ve que está caído. De este modo los siguientes µs ni lo intentan. También hay una política de recuperación para volver a dejar el acceso en estado abierto.
**Bulkheads:** no está como tal en el µs que llama, sino en medio, entre los µs. Se pone delante para no tumbar el µs en caso de muchas llamadas. Son barreras o protecciones que se sitúan entre los componentes y protegen sus recursos de manera que el fallo de un componente no afecte a otro: cuotas a nivel de CPU/RAM/persistencia; cuotas de uso de APIs; rate limit peticiones (DoS).
**Idempotencia:** es la propiedad de realizar una acción varias veces y conseguir el mismo resultado que se obtendría si se realizase una sola vez. Relacionado con el patrón Event Sourcing. Al haber tantos µs es posible que se dupliquen eventos. El productor genera un evento y lo mete en una cola para notificar a los consumidores. Si el evento se duplica, los consumidores no lo procesan 2 veces. Apache Kafka (Gestor de colas).

**Escalado**

**Vertical:** se trata de mejorar los recursos: más memoria, más CPU, más almacenamiento, más ancho de banda.
**Horizontal:** crecemos en instancias; podemos lanzar nuevos pods o servicios a demanda y bajarlos cuando la demanda ha bajado; también podemos crecer en workers, lo que me permite crecer en instancias.
**Particionar por datos:** distribuimos carga de trabajo haciendo particiones sobre los datos: usando una dimensión Apellido, Geografía, etc.
**Descomposición Funcional:** extraemos una funcionalidad muy demandante a otro µs, para luego poder escalarlo.